

1. PYTHON FOR ML/DL – CORE Q&A (BASIC → ADVANCED)

PYTHON FUNDAMENTALS (ML CONTEXT)

Q1. Why is Python preferred for Machine Learning?

Answer:

- Simple syntax → faster development
- Rich ecosystem (NumPy, Pandas, Scikit-learn, TensorFlow, PyTorch)
- Strong community support
- Easy integration with data pipelines

Python is widely used due to its flexibility and strong ML libraries ecosystem.

Q2. Difference between `is` and `==`

Answer:

- `==` → compares values
 - `is` → compares memory location
-

Q3. List vs Tuple (ML usage)

Answer:

- List → mutable (used for preprocessing)
 - Tuple → immutable (used for fixed configs)
-

Follow-ups:

- When would tuple be preferred in ML pipelines?
- Why immutability is useful?

DATA HANDLING (VERY IMPORTANT)

Q4. How do you load data in Python?

Answer:

```
import pandas as pd
df = pd.read_csv("data.csv")
```

Q5. Difference between Pandas and NumPy

Answer:

- NumPy → numerical operations (arrays)
 - Pandas → tabular data (DataFrame)
-

Q6. What is vectorization?

Answer:

- Performing operations on arrays without loops
 - Faster due to optimized C backend
-

Follow-ups:

- Why loops are slow in Python?
 - Example of vectorization vs loop?
-

FUNCTIONS & PERFORMANCE

Q7. What are Python generators?

Answer:

- Yield values one at a time
- Memory efficient for large datasets

Q8. What are decorators?

Answer:

- Modify function behavior without changing code
 - Used for logging, timing, etc.
-

Q9. What is list comprehension?

Answer:

- Compact way to create lists
 - Used in preprocessing pipelines
-

Follow-ups:

- Difference between generator vs list?
 - When NOT to use list comprehension?
-

OBJECT-ORIENTED PYTHON (ML PROJECTS)

Q10. Why OOP in ML?

Answer:

- Modular code
 - Reusable pipelines
 - Easy experimentation
-

Q11. What is `__init__`?

Answer:

- Constructor method for class initialization

Follow-ups:

- How do you design ML pipeline classes?
- Difference between class vs function approach?

ML-SPECIFIC PYTHON QUESTIONS

Q12. How do you split data?

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y)
```

Q13. How do you handle missing values?

```
df.fillna(df.mean())
```

Q14. How do you train a model?

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
```

Q15. How do you evaluate a model?

```
from sklearn.metrics import accuracy_score
accuracy_score(y_test, y_pred)
```

Follow-ups:

- How do you create a pipeline?
- Why scaling is required?

DEEP LEARNING (PYTHON)

Q16. TensorFlow vs PyTorch

Answer:

- TensorFlow → production ready
 - PyTorch → research friendly
-

Q17. How do you build a simple ANN?

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(64, activation='relu'),
    Dense(1, activation='sigmoid')
])
```

Q18. What is batching?

Answer:

- Training data in chunks
 - Improves memory efficiency
-

Q19. What is DataLoader (PyTorch)?

Answer:

- Handles batching, shuffling, parallel loading
-

Follow-ups:

- Why GPU is needed?
 - How do you debug training?
-
-

2. MOCK INTERVIEW SCRIPT (PYTHON FOR ML/DL)

ROUND 1: FUNDAMENTALS

Q1. Explain your ML workflow in Python

Expected Answer:

- Load data (Pandas)
 - Preprocess
 - Feature engineering
 - Train model
 - Evaluate
-

Follow-ups:

- Where do you spend most time?
 - Biggest bottleneck?
-

ROUND 2: CODING + OPTIMIZATION

Q2. Write code to normalize data

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Follow-ups:

- Why scaling is important?
 - When not required?
-

ROUND 3: DEBUGGING

Q3. Your model training is slow. What will you do?

Expected Answer:

- Vectorization
 - Use NumPy
 - Reduce features
 - Use GPU
-

Follow-ups:

- How do you profile Python code?
 - What is bottleneck identification?
-

ROUND 4: ADVANCED

Q4. How do you handle large datasets in Python?

Expected Answer:

- Generators
 - Chunking
 - Dask / PySpark
-

Follow-ups:

- When Pandas fails?
 - Difference vs Spark?
-
-

3. CASE STUDY (AMAZON-STYLE)

CASE 1: BUILD ML PIPELINE IN PYTHON

Problem:

Predict customer churn

Approach:

1. Data Loading

```
df = pd.read_csv("data.csv")
```

2. Preprocessing

- Handle missing
- Encoding

3. Model

```
model.fit(X_train, y_train)
```

Follow-ups:

- How do you modularize code?
 - How do you track experiments?
-
-

CASE 2: LARGE DATA PROCESSING

Problem:

Dataset = 10GB

Approach:

- Use chunking:

```
for chunk in pd.read_csv("data.csv", chunksize=10000):  
    process(chunk)
```

Follow-ups:

- Why not load entire dataset?
 - Alternative tools?
-
-

CASE 3: REAL-TIME PREDICTION SYSTEM

Problem:

Deploy ML model API

Approach:

- Save model:

```
import pickle  
pickle.dump(model, open("model.pkl", "wb"))
```

- Load + serve via Flask
-

Follow-ups:

- How do you handle latency?
 - How do you scale?
-
-

CASE 4: DL MODEL TRAINING PIPELINE

Problem:

Train image classifier

Approach:

- Load data
 - Data augmentation
 - Train CNN
-

Follow-ups:

- How do you avoid overfitting?
 - How do you monitor training?
-
-

CASE 5: MEMORY OPTIMIZATION

Problem:

Code running out of memory

Approach:

- Generators
 - Delete unused variables
 - Use sparse data
-

Follow-ups:

- What is garbage collection?
 - Difference between shallow vs deep copy?
-
-

GOLDEN FRAMEWORK (PYTHON ML/DL)

Candidate should ALWAYS follow:

1. Problem
 2. Data handling (Pandas/NumPy)
 3. Model building
 4. Optimization (vectorization)
 5. Deployment
-

WHAT INTERVIEWERS CHECK

- Python fundamentals
 - Library knowledge
 - Code efficiency
 - Debugging ability
 - Real-world ML usage
-

HIGH-VALUE TIP

Strong candidates:

- Write **clean, modular code**
- Avoid loops → prefer vectorization
- Know **why** not just **how**

ML interviews heavily test both **Python fundamentals + applied ML usage together**, not separately.
